

预处理的智慧

前缀和、差分与双指针 · 7题 · C++ & Python 双语对照

小鲁上节课学会了排序——把混乱变有序。这节课要学的是：有了有序性或预处理之后，如何以光速回答问题。小华说："排序之后数组有序了，二分查找 $O(\log n)$ ；但如果你经常要查区间和呢？每次都要 $O(n)$ 累加还是太慢——所以有了前缀和， $O(1)$ 回答。有了前缀和就有差分——修改一个区间只需两次操作。有了滑动窗口就有双指针——两个指针各走一遍解决 $O(n^2)$ 的问题。这些技巧的核心思想都是：预处理和挖掘有序结构。"

本章角色

-  小鲁 (进阶者) · 学习用预处理换取查询速度——"计算一次，查询千次"
-  小华 (算法队长) · 画图解释二维前缀和的容斥原理、差分矩阵的四角操作
-  小栋 (工程师) · 带真实场景来问：大数据量的区间查询和快速修改

前缀和·差分·双指针 核心模板

- 一维前缀和： $s[i]=s[i-1]+a[i]$, 查询 $[l,r]=s[r]-s[l-1]$
- 二维前缀和： $s[i][j]=s[i-1][j]+s[i][j-1]-s[i-1][j-1]+a[i][j]$ (容斥模式)
- 一维差分： $b[l]+=c$, $b[r+1]-=c$ (区间修改 $O(1)$)
- 二维差分：正角 $(x1,y1)+c$, $(x2+1,y2+1)+c$; 负角 $(x1,y2+1)-c$, $(x2+1,y1)-c$
- 滑动窗口：维护 $[l,r]$ 区间，while条件移动l，答案不断更新
- 对撞指针：i从左向右，j从右向左，根据条件判断移动哪个

NQ113 前缀和 AcWing 795

小栋拿着一个巨大的数组找到小鲁："我要查询100000次——每次求第l到第r个元素的和。直接每次循环累加的话要 $100000 \times 50000 = 50$ 亿次操作， $O(n \times q)$ 太慢了！"小华接过键盘："前缀和——预处理一个前缀数组 $s[i]$ =前i个元素的和，查 $[l,r]$ 的和只需 $s[r]-s[l-1]$ ， $O(1)$ 时间。用 $O(n)$ 的预处理换 $O(1)$ 的查询——这就是'预处理思维'的第一课。"

输入 第一行n和m，第二行n个整数，然后m行每行l r。

输出 每行输出 $a[l..r]$ 的和。

范围 $1 \leq n, m \leq 100000$

输入	输出
5 3	3
2 1 3 6 4	6
1 2	10
1 3	
2 4	

思路 预处理前缀数组: $s[i]=s[i-1]+a[i]$ (1-indexed)。查询 $[l,r]=s[r]-s[l-1]$, $O(1)$ 。Python用accumulate。C++注意使用1-indexed数组简化边界处理。前缀和的核心思想: 把重复计算变成一次计算+多次查表。

技巧 用1-indexed数组避免l-1越界—— $s[0]=0$ 。Python: $s=\text{list}(\text{accumulate}([0]+\text{arr}))$ 。前缀和可以扩展到异或和、乘积和、最大值/最小值 (需要线段树)。理解预处理思维是算法优化的核心。

C++

```
#include <iostream>

using namespace std;
const int N = 1000007;
int n, m;

int a[N], s[N];

int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i++)
        scanf("%d", &a[i]);
    s[0] = 0; //刻意初始化s[0]为0, 此句可以不写, 写也无妨
    for (int i = 1; i <= n; i++)
        s[i] = s[i - 1] + a[i]; //打表法
    //m次询问
    while (m--)
    {
        int l, r;
        scanf("%d%d", &l, &r);
        printf("%d\n", s[r] - s[l - 1]); //求区间和, 直接从s
        中读取数值
    }

    return 0;
}
```

Python

```
# Python solution
```

NQ114 子矩阵的和 AcWing 796

"前缀和能不能推广到二维?"小鲁问。"当然!"小华在白板上画出矩阵, "二维前缀和: $s[i][j]=s[i-1][j]+s[i][j-1]-s[i-1][j-1]+a[i][j]$ ——包含关系的加减。查询子矩阵 $(x1,y1)$ 到 $(x2,y2)$ 的和= $s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1]$ 。看这个'容斥'模式——包含-排除+补回。"

输入 第一行 n,m,q 。然后 $n \times m$ 矩阵。然后 q 行每行 $x1\ y1\ x2\ y2$ 。

输出 每行子矩阵元素和。

范围

$1 \leq n, m \leq 1000, 1 \leq q \leq 200000$

输入	输出
3 4 3	17
1 7 2 4	27
3 6 2 8	21
2 1 2 3	
1 1 2 2	
2 1 3 4	
1 3 3 4	

思路 二维前缀和：预处理 $s[i][j]$ 包含从 $(1,1)$ 到 (i,j) 的矩形和。公式如上述。查询子矩阵和=大矩形减两翼加回重叠部（容斥原理）。 $O(nm)$ 预处理 $\rightarrow O(1)$ 每次查询。Python同样用accumulate思想。

技巧 注意预处理和查询的加减符号—— $s[i][j]=s[i-1][j]+s[i][j-1]-s[i-1][j-1]+a[i][j]$ （包含关系）。查询时=大矩形-上矩形-左矩形+左上矩形（容斥）。1-indexed简化边界。理解这个模式后三维前缀和也是同样的加减法。

C++

```
#include <iostream>
using namespace std;
const int N = 1007;
int n, m, q;
int a[N][N], s[N][N]; //子矩阵

int main()
{
    //读入矩阵
    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            scanf("%d", &a[i][j]);

    s[0][0] = 0; //刻意初始化s[0]为0，此句可以不写，写也无妨
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j]; //打表法
    //q次询问
    while (q--)
    {
        int x1, x2, y1, y2;
        scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
        //求子矩阵和，直接从s中读取数值
        printf("%d\n", s[x2][y2] - s[x2][y1-1] - s[x1-1][y2] + s[x1-1][y1-1]);
    }
    return 0;
}
```

Python

Python solution

"前缀和是前缀→查询。那它的逆运算是什么？"小华问。"如果我需要对数组的 $[l,r]$ 区间每个元素都加 c ，怎么最快？"小鲁想了想："差分！构造 b 数组使 a 是 b 的前缀和。对 $[l,r]$ 加 c 只需 $b[l]+=c$, $b[r+1]-=c$ ——两次操作！对整个区间每个元素加同一个值——这是差分最经典的用法。 $O(1)$ 修改， $O(n)$ 还原。"

输入 第一行 n,m 。第二行 n 个整数。然后 m 行每行 $l\ r\ c$ 。

输出 所有操作后的数组。

范围 $1 \leq n,m \leq 100000$

输入

```
6 3
1 2 2 1 2 1
1 3 1
3 5 1
1 6 1
```

输出

```
3 4 5 3 4 2
```

思路 差分数组 b : $b[i]=a[i]-a[i-1]$ (用 $a[i]$ 的差分量), 则 $a[i]=b[1]+...+b[i]$ 。区间 $[l,r]$ 加 c : $b[l]+=c$, $b[r+1]-=c$ 。最后做前缀和还原。这是'区间修改→点查询'的最优方案—— $O(1)$ 修改 $O(n)$ 还原。

技巧 差分=前缀和的逆运算。核心操作: $b[l]+=c$, $b[r+1]-=c$ 。Python实现可直接用数组操作。理解差分的妙处: 两次修改影响整个区间。竞赛中差分和前缀和是黄金搭档——90%的区间问题都可以用这俩之一解决。

C++

```
#include <iostream>
using namespace std;
const int N = 1000007;
int n, m;
int a[N], b[N];

//差分的插入公式
void insert(int l, int r, int c)
{
    b[l] += c;
    b[r + 1] -= c;
}

int main()
{
    cin >> n >> m;
    //读入数组
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    //构造差分数组b
    for (int i = 1; i <= n; i++)
        insert(i, i, a[i]);
    //输出查询结果
    while (m--)
    {
        int l, r, c;
        cin >> l >> r >> c;
        insert(l, r, c);
    }
    //重新复原a数组(前缀和)
    for (int i = 1; i <= n; i++)
        a[i] = a[i - 1] + b[i];
    //输出a
    for (int i = 1; i <= n; i++)
        cout << a[i] << " ";
    cout << endl;
    return 0;
}
```

Python

Python solution

NQ116 差分矩阵 AcWing 798

"一维差分搞定了——二维呢?"小栋继续追问。小华画出矩阵:"二维差分——对子矩阵(x1,y1)到(x2,y2)加c只需四步: $b[x1][y1] += c$, $b[x2+1][y1] -= c$, $b[x1][y2+1] -= c$, $b[x2+1][y2+1] += c$ 。注意四个角——加主对角减副对角。最后做二维前缀和还原。两次操作,影响整个子矩阵——这就是数学建模的优雅。"

输入 第一行n,m,q。然后n×m矩阵。然后q行每行x1 y1 x2 y2 c。

输出 所有操作后的矩阵。

范围 $1 \leq n, m \leq 1000, 1 \leq q \leq 100000$

输入

```
3 4 3
1 2 2 1
3 2 2 1
1 1 1 1
```

输出

```
2 3 4 1
4 3 4 1
2 2 2 2
```

```
1 1 2 2 1
1 3 2 3 2
3 1 3 4 1
```

思路 二维差分： $b[i][j]$ 使 $a[i][j]=b[1..i][1..j]$ 的和。子矩阵加 c 的四步操作如上述——两个正角 $(x1,y1$ 和 $x2+1,y2+1)$ 两个负角 $(x2+1,y1$ 和 $x1,y2+1)$ 。最后二维前缀和还原。 $O(1)$ 修改 $\rightarrow O(nm)$ 还原。

技巧 记忆四角公式：正正负负。左上角 $(x1,y1)+c$ ，右上角 $(x1,y2+1)-c$ ，左下角 $(x2+1,y1)-c$ ，右下角 $(x2+1,y2+1)+c$ 。二维差分比一维多了一对操作。这是图像处理中滤镜效果的基础——用极少操作影响大区域。

C++

```

#include <iostream>
using namespace std;
const int N = 1007;
int n, m, q;
int a[N][N], b[N][N]; //子矩阵
//二维差分核心操作
void insert(int x1, int y1, int x2, int y2, int c)
{
    b[x1][y1] += c;
    b[x1][y2 + 1] -= c;
    b[x2 + 1][y1] -= c;
    b[x2 + 1][y2 + 1] += c;
}
int main()
{
    //读入矩阵
    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            scanf("%d", &a[i][j]);

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            insert(i, j, i, j, a[i][j]); //构造二维差分矩阵

    //q次询问
    while (q--)
    {
        int x1, x2, y1, y2, c;
        scanf("%d%d%d%d%d", &x1, &y1, &x2, &y2, &c);
        //差分核心操作
        insert(x1, y1, x2, y2, c);
    }
    //重建a[i][j],即求二维前缀和
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            a[i][j] = a[i - 1][j] + a[i][j - 1] - a[i - 1][j - 1] + b[i][j];
    //输出a[i][j]
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
            cout << a[i][j] << " ";
        cout << endl;
    }
    return 0;
}

```

Python

Python solution

NQ117 最长连续不重复子序列 AcWing 799

"给一个数组，找最长的一段，里面的数都不重复。"小华说，"暴力三重循环 $O(n^3)$ 太慢。优化——用双指针！维护窗口 $[l, r]$ ，用计数数组记录窗口内每个数出现多少次。当 $a[r]$ 重复时，不断 $l++$ 直到消除重复。双指针的复杂度是 $O(2n)=O(n)$ ——每个元素最多进窗口一次，出窗口一次。"

输入

第一行 n ，第二行 n 个整数。

输出

最长连续不重复子序列的长度。

范围

$1 \leq n \leq 100000$

输入

5
1 2 2 3 5

输出

3

思路 双指针滑动窗口：维护 $[l, r]$ 区间无重复。遍历 r ，每次 $\text{cnt}[a[r]]++$ 。若 $\text{cnt}[a[r]] > 1$ 说明有重复， $\text{while}(\text{cnt}[a[r]] > 1) \text{cnt}[a[l]]--, l++$ 。答案 $= \max(\text{ans}, r-l+1)$ 。时间复杂度 $O(n)$ ——每个元素最多入窗一次离开一次。

技巧 双指针核心：两指针都只向前移动，从不后退——保证 $O(n)$ 。重复检测用计数数组作哈希（值域小时）。Python用字典记录上次出现位置可优化到只移一次。双指针是处理'连续子数组'类问题的常用武器。

C++

```
#include <iostream>
using namespace std;
const int N=100007;
int num[N],visited[N];

int main(){
    int n;
    cin>>n;
    for (int i=0; i<n; i++ ) cin>>num[i]; //读入n个数
    //双指针扫描
    int res=0; //初始化为最小值
    for (int i=0, j=0; i<n; i++){
        visited[num[i]]++;
        while(j<=i && visited[num[i]]>1){
            visited[num[j]]--;
            j++; //去掉重复数
        }
        res= max(res, i-j+1); //j--i 之间的数字个数
    }
    cout<<res<<endl;
    return 0;
}
```

Python

```
# Python solution
```

NQ118 数组元素的目标和 AcWing 800

"两个升序数组A和B，找一对 $a[i]+b[j]=x$ 。" "如果暴力双循环 $O(n \times m)$ 太慢——但既然两个数组都是升序的....."小鲁已经有了思路，"用双指针！ i 从A的开头开始向右走， j 从B的末尾开始向左走。 $a[i]+b[j]$ 比 x 大就 $j--$ ，比 x 小就 $i++$ 。对撞双指针——各自走一遍， $O(n+m)$ 。"

输入

第一行 n, m, x 。第二行 n 个升序整数(A)。第三行 m 个升序整数(B)。

输出

i 和 j (0-indexed)。

范围

$1 \leq n, m \leq 100000$ 。保证有唯一解。

输入

输出

```
4 5 6
1 2 4 7
3 4 6 8 9
```

```
1 1
```

思路 对撞双指针：i=0从左，j=m-1从右。当a[i]+b[j]>x则j--（太大，缩右边的），当a[i]+b[j]

技巧 双指针三种模式：滑动窗口（同向）、对撞指针（反向）、快慢指针。本题是对撞指针——从两端向中间逼近。抓住while循环的两个关键：条件(a[i]+b[j] vs x)和移动方向(谁移动)。

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
const int N=100007;
int n,m,x;
int numA[N],numB[N];

int main(){
    //读入n,m,x其中x为目标和
    scanf("%d%d%d",&n,&m,&x);
    for (int i=0; i<n; i++) scanf("%d",&numA[i]);
    for (int j=0; j<m; j++) scanf("%d",&numB[j]);
    //双指针查找和

    for (int i=0, j= m-1; i<n; i++){
        //check(i,j)
        while (j>=0 && numA[i]+numB[j]>x) j--;
        if (numA[i]+numB[j]==x)//判断是否和相等
        {
            printf("%d %d\n",i,j);
            break;
        }
    }
    return 0;
}
```

Python

```
# Python solution
```

NQ119 判断子序列 AcWing 2816

"最后一个双指针问题：判断a是不是b的子序列——即a的元素按相同顺序出现在b中（不一定连续）。"小华说，"双指针i指a、j指b。遍历b的每个元素，如果b[j]==a[i]就i++。最后如果i==n说明a的所有元素都按顺序找到了。这个'匹配式'双指针——j不停走，i只在匹配时走——O(n+m)。"

输入 第一行n,m。第二行n个整数(a)。第三行m个整数(b)。

输出 是子序列输出Yes，否则No。

范围 $1 \leq n \leq m \leq 100000$

输入

```
3 5
1 3 5
1 2 3 4 5
```

输出

```
Yes
```

思路 匹配双指针：i=0指a，j=0指b。遍历j，if b[j]==a[i]: i++。最后i==n则匹配完成。j一直移动，i只在匹配成功时前移。O(n+m)时间O(1)空间。此模式适用于有序序列的判断和验证问题。

技巧 判断子序列是双指针的经典入门题——理解'只有匹配时才移动i'的节奏。这个模式也是KMP算法的前身——都是字符串/序列匹配，但双指针版更简单直白。竞赛中大量'验证是否满足某约束'的题都在用这个思路。

C++

```
#include <iostream>
#include <cstdio>
using namespace std;
const int N = 1e5+7;//我的风格
int a[N],b[N];

int main()
{
    int n,m;
    scanf("%d%d",&n,&m);
    for (int i = 0; i<n; i++) scanf("%d",&a[i]);
    for (int i = 0; i<m; i++) scanf("%d",&b[i]);

    int p = 0;//指向a数组的指针
    for (int i = 0; i < m; i++)
    {
        if (p < n && a[p] == b[i]) p++;
    }
    //完全匹配
    if (p==n) puts("Yes");
    else puts("No");

    return 0;
}
```

Python

```
# Python solution
```

本章知识点总结

- 前缀和 (1D/2D)：O(n)预处理→O(1)查询。"重复查询→预处理"思维的典型应用
- 差分 (1D/2D)：O(1)区间修改→O(n)还原。"修改操作远多于查询"场景的最优方案
- 双指针三模式：滑动窗口、对撞指针、匹配指针——核心是不回溯，O(n)解决O(n²)问题
- 厦大精神：自强不息。这些预处理技巧，在真实工程中（大数据、图像处理）同样大放异彩